

Compute Services Quick Reference

EC2 (Elastic Compute Cloud)

Instance Types (Know the families!)

Family	Type	Use Case	Example
General Purpose	T3, T3a, T4g, M5, M6i	Balanced compute/memory/network	Web servers, small DBs
Compute Optimized	C5, C6i, C7g	High-performance processors	Batch processing, gaming, HPC
Memory Optimized	R5, R6i, X2idn, z1d	High memory-to-CPU ratio	In-memory databases, big data
Storage Optimized	I3, I4i, D2, D3	High sequential read/write	Data warehouses, log processing
Accelerated Computing	P3, P4, G4, Inf1	GPU/FPGA workloads	ML training, graphics rendering

Placement Groups

Type	Use Case	Benefit	Limitation
Cluster	HPC, low-latency apps	Single AZ, 10 Gbps network	Single point of failure
Spread	Critical instances, HA	Different hardware, up to 7 per AZ	Limited scale
Partition	Large distributed systems (Hadoop, Cassandra)	Groups on separate racks	Up to 7 partitions per AZ

Pricing Models

Model	Discount	Use Case	Commitment
On-Demand	0% (baseline)	Unpredictable workloads, testing	None
Reserved (Standard)	40-60%	Steady-state, predictable	1 or 3 years
Reserved (Convertible)	30-45%	Change instance type later	1 or 3 years

Model	Discount	Use Case	Commitment
Savings Plans	40-60%	Flexible across instance families/regions	1 or 3 years
Spot	Up to 90%	Fault-tolerant, flexible	Can be interrupted
Dedicated Hosts	Varies	Compliance, BYOL software	Per-host billing

Auto Scaling Policies

Type	How It Works	Use Case
Target Tracking	Maintain target metric (e.g., 70% CPU)	LEAST operational overhead, general use
Step Scaling	Scale based on CloudWatch alarms + steps	Fine-grained control, multiple thresholds
Scheduled Scaling	Scale at specific times	Predictable traffic patterns
Predictive Scaling	ML-based forecasting	Recurring traffic patterns

Key Exam Tips

- **EBS-backed instances:** Can be stopped (data persists)
- **Instance store:** Ephemeral, data lost on stop/terminate (high IOPS)
- **T3 instances:** Burstable performance, CPU credits (good for variable workloads)
- **Tenancy:** Default (shared) | Dedicated (hardware isolation) | Dedicated Host (BYOL)
- **User Data:** Script runs on FIRST LAUNCH only (unless configured otherwise)
- **Metadata:** Access via <http://169.254.169.254/latest/meta-data/>
- **Hibernate:** Saves RAM to EBS, faster startup than stop/start

Lambda

Key Specs (MEMORIZE!)

Attribute	Limit	Notes
Timeout	15 minutes max	Default is 3 seconds
Memory	128 MB - 10,240 MB (10 GB)	CPU scales with memory

Attribute	Limit	Notes
Deployment package	50 MB (zipped), 250 MB (unzipped)	Use layers for dependencies
Ephemeral storage (/tmp)	512 MB - 10,240 MB	Temporary, not persistent
Environment variables	4 KB total	Use Parameter Store/Secrets Manager for larger configs
Concurrent executions	1,000 (account limit, can increase)	Can set reserved concurrency

Invocation Types

Type	Behavior	Use Case
Synchronous	Waits for response	API Gateway, ALB, direct invoke
Asynchronous	Returns immediately, retries on failure	S3 events, SNS, EventBridge
Event Source Mapping	Lambda polls the source	SQS, Kinesis, DynamoDB Streams

Lambda Pricing

- **Requests:** \$0.20 per 1M requests
- **Duration:** \$0.0000166667 per GB-second
- **Free Tier:** 1M requests + 400,000 GB-seconds per month

Cold Starts

- **Cause:** First invocation or after idle period
- **Duration:** 100ms - several seconds (depends on runtime, VPC)
- **Solutions:**
 - Provisioned Concurrency (keeps functions warm, costs more)
 - Smaller deployment packages
 - Avoid VPC if not needed (VPC adds latency)

Lambda@Edge

- Run Lambda at CloudFront edge locations
- Use cases: Header manipulation, A/B testing, authentication
- Limitations: 5 second timeout, limited memory

Key Exam Tips

- **Lambda is NOT for:** Long-running tasks (> 15 min), stateful applications, real-time latency requirements (< 10ms)
- **Lambda IS for:** Event-driven, short-lived, stateless, microservices
- **VPC Lambda:** Can access private resources but adds cold start latency
- **Layers:** Share code/libraries across functions (max 5 layers, 250 MB total)
- **Dead Letter Queue:** SQS or SNS for failed async invocations
- **Destinations:** Route execution results (success/failure) to targets

Container Services

ECS (Elastic Container Service)

Launch Type	Description	Use Case	Pricing
EC2	Run containers on EC2 instances you manage	Full control, existing EC2 reservations	EC2 instance costs
Fargate	Serverless, AWS manages infrastructure	No server management, variable workloads	Per vCPU/memory/second

Key Concepts:

- **Task Definition:** Blueprint for your application (like Dockerfile)
- **Service:** Maintains desired count of tasks, integrates with ALB
- **Cluster:** Logical grouping of tasks/services
- **Task:** Running instance of task definition

EKS (Elastic Kubernetes Service)

- Managed Kubernetes control plane
- Use cases: Kubernetes expertise, multi-cloud, complex orchestration
- More expensive than ECS (control plane costs + EC2/Fargate)
- **Fargate for EKS:** Serverless Kubernetes pods

Comparison

Feature	ECS	EKS
Learning curve	Low	High (Kubernetes knowledge)
AWS integration	Native, tight integration	Good integration
Multi-cloud	AWS only	Portable (Kubernetes)
Cost	Lower	Higher (control plane cost)

Feature	ECS	EKS
Use case	AWS-native containerized apps	Kubernetes requirement, multi-cloud

Elastic Beanstalk

- **What:** PaaS for deploying web apps (Java, .NET, Node.js, Python, Ruby, Go, Docker)
- **You control:** Application code
- **AWS controls:** Provisioning, load balancing, scaling, monitoring
- **Pricing:** No additional charge (pay for underlying resources)
- **Deployment options:** All at once, Rolling, Rolling with batch, Immutable, Blue/Green
- **Use case:** Quick deployment without managing infrastructure, LEAST operational overhead

Batch

- **What:** Run batch computing jobs (100s-1000s of jobs)
- **Features:** Job scheduling, manages compute resources, integrates with Spot
- **Use case:** Financial modeling, genomics, video rendering
- **vs Lambda:** Batch for long-running jobs (hours/days), Lambda for short tasks (<15 min)

Exam Pattern Recognition

"LEAST operational overhead" → Choose:

1. Lambda (serverless)
2. Fargate (no server management)
3. Elastic Beanstalk (managed platform)

"MOST cost-effective" → Choose:

1. Spot Instances (fault-tolerant workloads)
2. Lambda (pay per use, variable workloads)
3. Auto Scaling (scale down when idle)

"Long-running, stateful processing" → Choose:

1. EC2 (not Lambda - 15 min limit)
2. ECS on EC2 (if containerized)

"High-performance computing, low latency" → Choose:

1. EC2 Cluster Placement Groups

2. Instance store (high IOPS, ephemeral)

"Batch processing, flexible timing" → Choose:

1. Spot Instances (up to 90% savings)
2. AWS Batch (job scheduling)